



9.1. Operations on Stacks

[Previous Section:](#)

 [9. Stacks](#)

Contents:

[1. Basic Operations](#)

[1.1. Stacks Using Arrays](#)

[1.2. Stacks Using Linked Lists](#)

[2. Reversing a String Using a Stack](#)

[3. Two Stacks in One Array](#)

[Summary](#)

[References](#)

Now that we understand what a stack is, the next question becomes: **How do we actually use one?**

A stack is not very useful if we can only look at it. We need operations that allow us to add elements, remove elements, inspect the top element, and determine whether the stack contains any data.

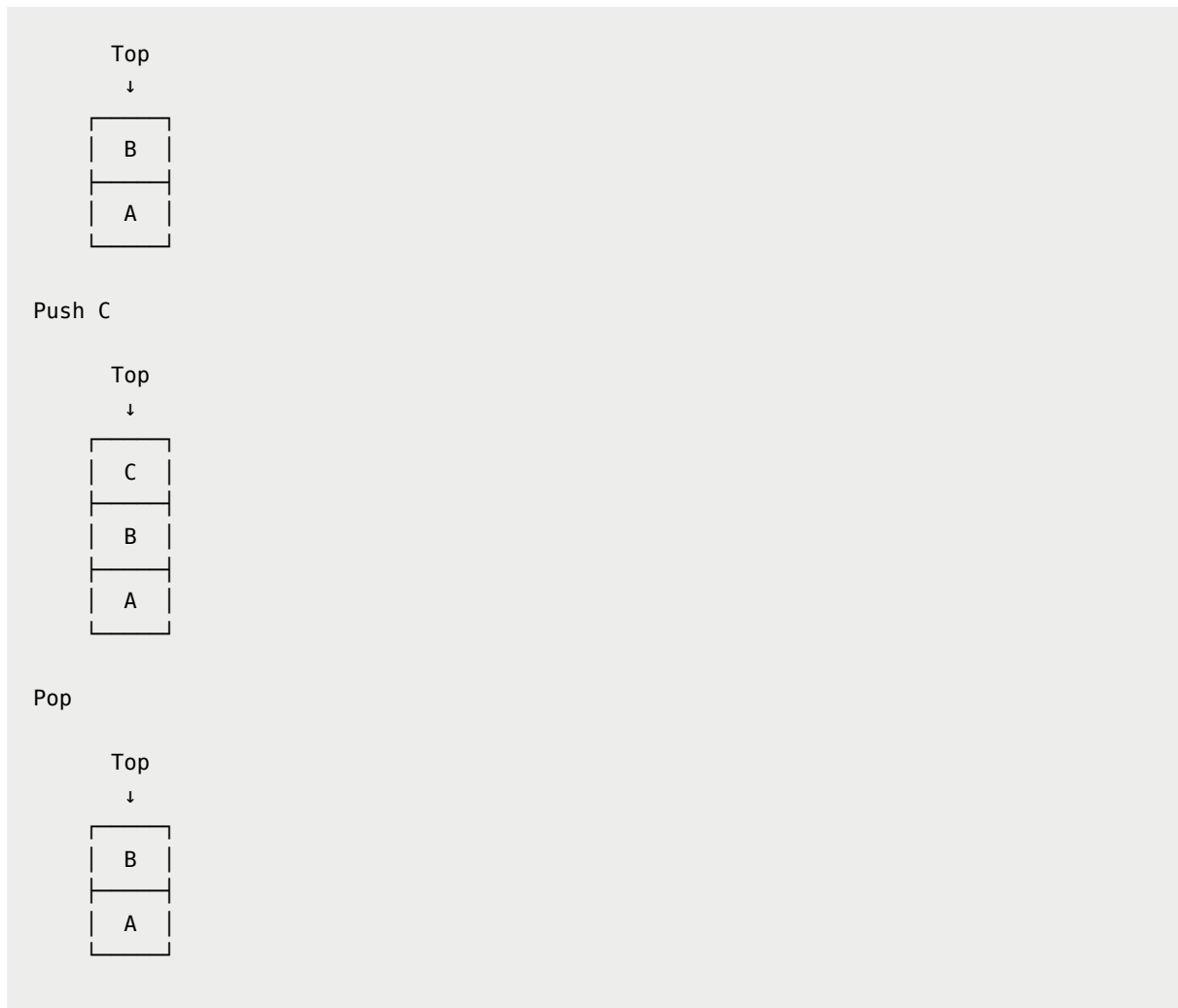
Remember that a stack follows the **Last In, First Out (LIFO)** principle.

Push A

Top



Push B



Every operation we perform on a stack revolves around its **Top** element.

In this section, we will explore the most common stack operations and learn how to implement stacks using both arrays and linked lists.

1. Basic Operations

To manipulate a stack, we need several fundamental operations.

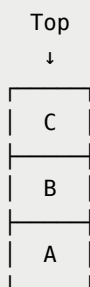
Operation	Description	Time Complexity
PUSH	Insert an element onto the stack	$O(1)$
POP	Remove the top element from the stack	$O(1)$
DISPLAY	Traverse and display all elements	$O(n)$

Operation	Description	Time Complexity
GET TOP (PEEK)	Return the top element without removing it	$O(1)$
IS EMPTY	Check whether the stack contains any element	$O(1)$

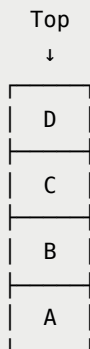
PUSH

The **Push** operation inserts a new element onto the top of the stack. Time Complexity: $O(1)$

Before Push(D)



After Push(D)

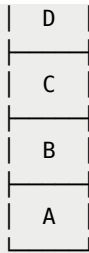


POP

The **Pop** operation removes the element currently located at the top. Time Complexity: $O(1)$

Before Pop()





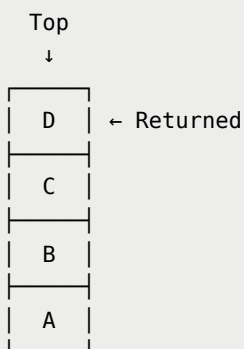
Removed: D

After Pop()



GET TOP (PEEK)

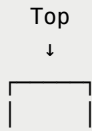
The **Peek** operation returns the top element without removing it. Time Complexity: $O(1)$



IS EMPTY

This operation determines whether the stack contains any elements. Time Complexity: $O(1)$

Stack:



Result: True

DISPLAY

Display traverses every element inside the stack. Time Complexity: $O(n)$

Top
↓

C -> B -> A -> None

1.1. Stacks Using Arrays

The simplest implementation of a stack uses a Python list. Instead of manually managing memory, we let Python dynamically resize the list whenever necessary.

```
class Stack:
    def __init__(self):
        self.items = []

    def is_empty(self): # Check if a stack is empty
        return len(self.items) == 0

    def display(self): # Traverse all nodes
        for i in range(len(self.items)):
            print(self.items[i], "->", end=" ")
        print()

    def push(self, data): # Append a value to the stack
```

```

        self.items.append(data)

    def pop(self): # Take the top value out of the stack
        if self.is_empty(): # Empty stack
            return None
        else:
            return self.items.pop()

    def get_top(self): # Get a value at the top of the stack
        if self.is_empty():
            return None
        else:
            return self.items[-1]

```

```

stack = Stack()

stack.push("A")
stack.push("B")
stack.push("C")

stack.display()

print("Top:", stack.get_top())

stack.pop()

stack.display()

```

A → B → C →

Top: C

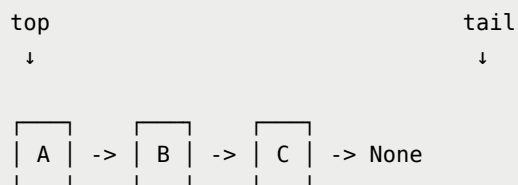
A → B →

1.2. Stacks Using Linked Lists

Although arrays work well, stacks can also be implemented using linked lists.

In this implementation, we maintain:

- `top` → beginning of the list
- `tail` → last element (actual stack top)



- Node Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

    def display(self):
        print(self.data, "->", end=" ")
```

- Stack Class

```
class Stack:
    def __init__(self):
        self.top = None
        self.tail = None

    def is_empty(self):
        return self.top is None

    def display(self):
        current = self.top

        while current:
            current.display()
            current = current.next
```

```

print()

def push(self, data):
    new_node = Node(data)

    if self.top is None:
        self.top = new_node
        self.tail = new_node
    else:
        self.tail.next = new_node
        self.tail = new_node

def pop(self):
    if self.is_empty():
        return None

    if self.top == self.tail:
        data = self.top.data
        self.top = None
        self.tail = None
        return data

    current = self.top

    while current.next != self.tail:
        current = current.next

    data = self.tail.data
    self.tail = current
    self.tail.next = None

    return data

def get_top(self):
    if self.is_empty():
        return None

```



```
return self.tail.data
```

Note: This implementation keeps the stack top at the tail of the linked list. While it works correctly, a more efficient implementation usually inserts and removes directly at the head to avoid traversal during the pop operation.

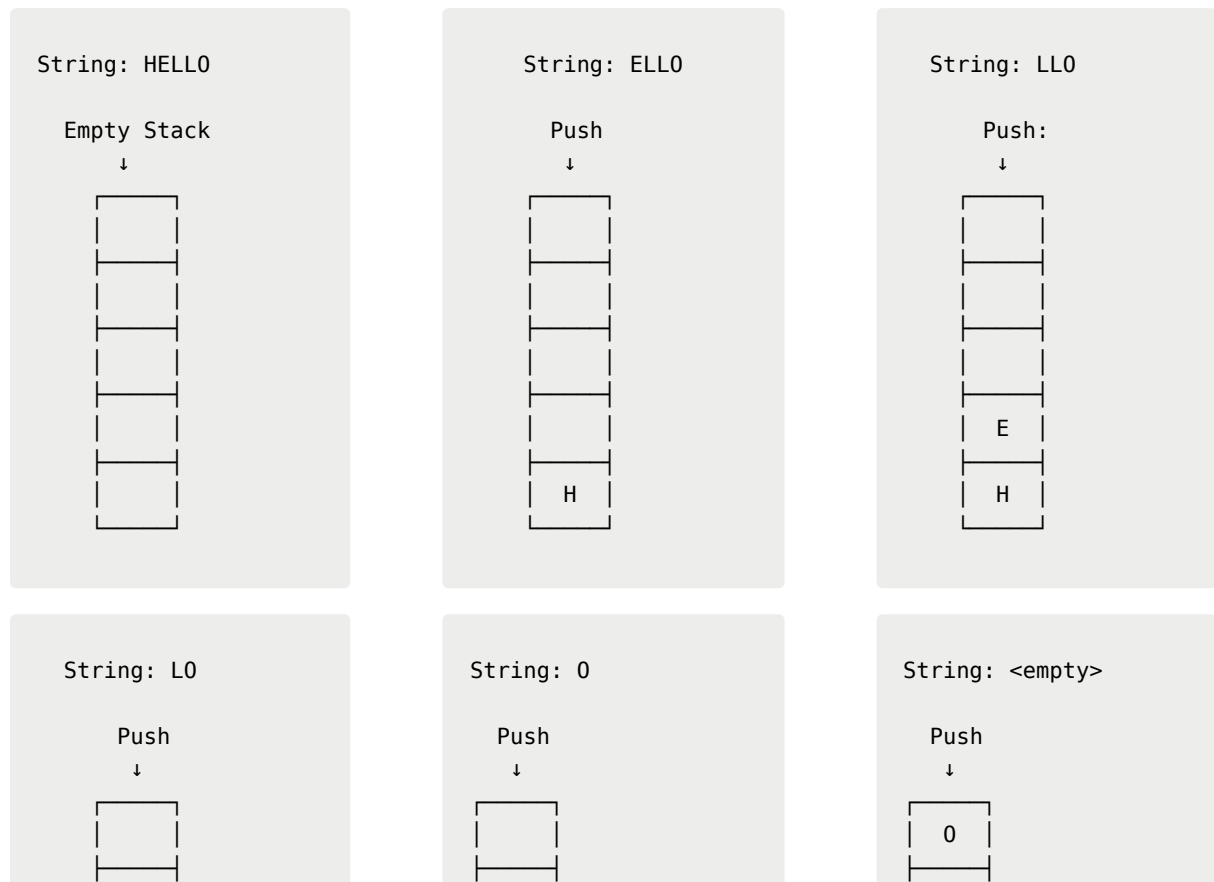
2. Reversing a String Using a Stack

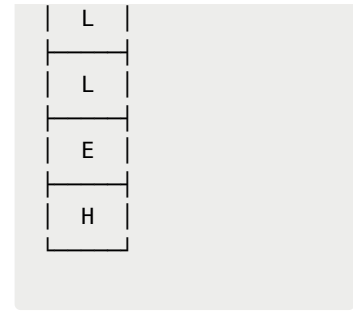
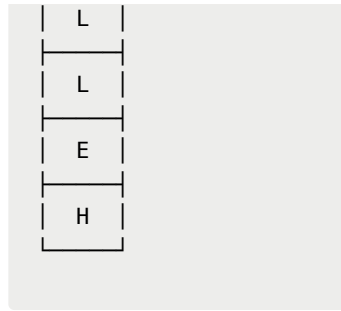
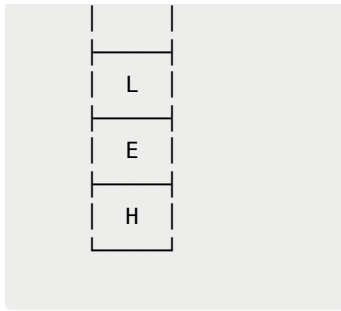
One of the classic applications of stacks is reversing strings. The idea is simple:

1. Push every character into the stack.
2. Pop every character from the stack.
3. Since stacks follow LIFO, the string comes out reversed.

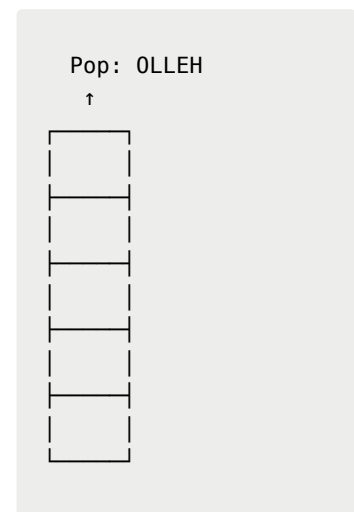
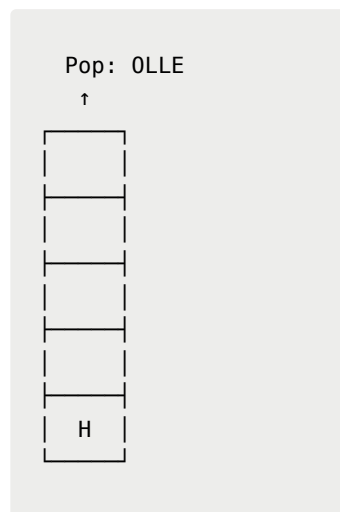
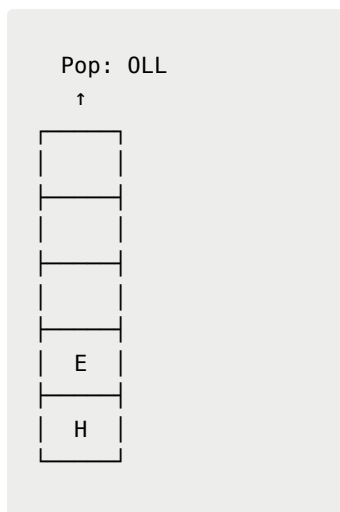
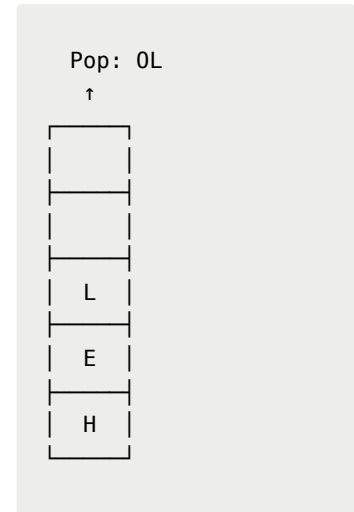
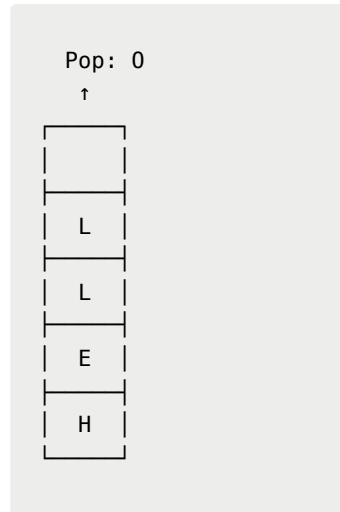
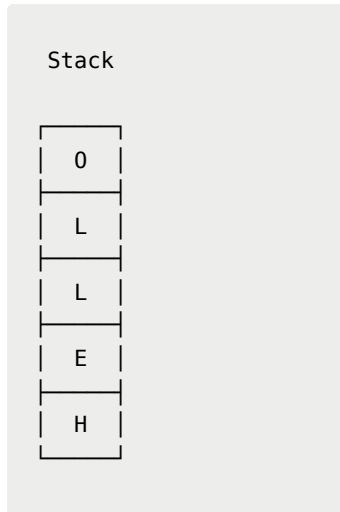
Example - Original String: HELLO

Push all characters:





Pop all characters:



Result: **OLLEH**

This demonstrates how a stack naturally reverses the order of data.

Implementation:

```
def reverse_string(text):
    stack = []

    for ch in text:
        stack.append(ch)

    reversed_text = ""

    while stack:
        reversed_text += stack.pop()

    return reversed_text
```

```
print(reverse_string("HELLO"))
```

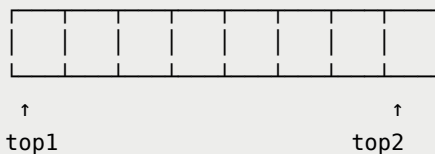
OLLEH

3. Two Stacks in One Array

Now let's explore an interesting optimization. Instead of allocating two separate arrays for two stacks, we can store both stacks inside a single array.

One stack grows from the left. The other stack grows from the right.

Initial State



After Several Insertions

Stack 1 grows →

Stack 2 grows ←



The stacks continue growing until the two pointers meet. This approach uses memory much more efficiently than maintaining two separate arrays.

Implementation:

```
class TwoStack:
    def __init__(self, n):
        self.size = n
        self.arr = [None] * n

        self.top1 = -1
        self.top2 = self.size

    def push1(self, x):
        if self.top1 < self.top2 - 1:
            self.top1 += 1
            self.arr[self.top1] = x
        else:
            raise Exception("Stack Overflow")

    def push2(self, x):
        if self.top1 < self.top2 - 1:
            self.top2 -= 1
            self.arr[self.top2] = x
        else:
            raise Exception("Stack Overflow")

    def pop1(self):
        if self.top1 >= 0:
            x = self.arr[self.top1]
            self.top1 -= 1
            return x
```

```

        raise Exception("Stack Underflow")

def pop2(self):
    if self.top2 < self.size:
        x = self.arr[self.top2]
        self.top2 += 1
        return x

        raise Exception("Stack Underflow")

def display(self):
    print("Stack 1:")
    for i in range(self.top1 + 1):
        print(self.arr[i], end=" -> ")
    print()

    print("Stack 2:")
    for i in range(self.size - 1,
                    self.top2 - 1,
                    -1):
        print(self.arr[i], end=" -> ")
    print()

```

```
ts = TwoStack(10)
```

```
ts.push1("A")
```

```
ts.push1("B")
```

```
ts.push1("C")
```

```
ts.push2("X")
```

```
ts.push2("Y")
```

```
ts.push2("Z")
```

```
ts.display()
```

Stack 1:

A → B → C →

Stack 2:

X → Y → Z →

Summary

Stacks provide a small set of operations, but those operations are powerful enough to solve many practical problems.

The most important operations are:

- **Push** → insert an element
- **Pop** → remove the top element
- **Peek** → inspect the top element
- **Display** → traverse the stack
- **Is Empty** → determine whether the stack contains data

We can implement stacks using either arrays or linked lists, and once implemented, stacks can be used to solve larger problems such as **string reversal**, **expression evaluation**, **recursion management**, and **syntax validation**.

We can even optimize memory usage by storing **two independent stacks inside a single array**, allowing both stacks to grow toward each other while

sharing the same storage space.

In the next section, we will explore more advanced applications of stacks and see how they are used to solve real-world computational problems.

References

https://drive.google.com/file/d/1yBJEh1C9V-nXZ7TF80JtWvozfaDYRq5B/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/basic-operations-in-stack-data-structure-with-implementations/>

Next Section:

 [9.2. Expression Evaluation using Stacks](#)